PDP Document Display Block

Map to PDP Document Display - Parent Component

1. Overview

The **PDP Document Display component** integrates an external Product Data Page (PDP) documents application into EDS. This external application provides document browsing and filtering capabilities based on product SKUs, document IDs, and document types.

External JavaScript application handles:

- Document fetching
- Filtering (e.g., manuals, specs, etc.)
- UI rendering

EDS acts as a **host layer**, responsible for:

- Capturing author input
- Loading the external script
- Passing configuration to the external app
- Providing a container for rendering

---

2. Current State

Existing Behavior

- Authors configure:
  - Product SKU
  - Document ID(s)
  - Document types (e.g., spec, manual)
- On component render:
  a. External JS is loaded:

```
/store/v2/documents/static/conditionalLoadAssets.js
```

  b. This script defines a global function:

```
window.documentsConfiguration()
```

  c. The component creates a configuration object (`pdpConfigObject`)
  d. Calls:

```
window.documentsConfiguration(pdpConfigObject)
```

- The external application:
  - Consumes the config
  - Fetches relevant document data
  - Renders a rich UI (filters, lists, etc.) inside a target container

---

3. EDS Approach

Block-Based Implementation

A dedicated EDS block will be created.

This block will:

1. **Read authored configuration**
   - SKU
   - Document IDs
   - Document types
2. **Create a container**
   - A `<div>` with a `containerId`
   - Used by the external app to render content
3. **Load external JS dynamically**
   - Path (configurable via central config):
   - External JS path can be managed via configuration

```
/store/v2/documents/static/conditionalLoadAssets.js
```

4. **Construct configuration object**

```
const config = {
  sku: "...",
  documentIds: [...],
  documentTypes: [...],
  containerId: "pdp-documents-123"
};
```

5. **Invoke external application**

```
window.documentsConfiguration(config);
```

Performance & Optimization (EDS Recommended Pattern)

To ensure optimal performance and alignment with EDS best practices:

- **Lazy Initialization using IntersectionObserver**
  - Defer loading of the external JS and initialization until the block is near or enters the viewport
  - Especially useful when the block is below the fold
- **Benefits**
  - Improves **Core Web Vitals**
  - Reduces initial page load time
  - Improves Lighthouse scores and perceived performance
- **Additional Considerations**
  - Ensure the external script is loaded **only once** (avoid duplicate loads)
  - Keep the EDS block lightweight and focused only on orchestration

---

4. Authoring Block

Block Name

`PDP Document Display`

Authoring Fields

Field Name	Description
SKU	Product SKU used to fetch documents
Document IDs	Specific document identifiers (optional)
Document Types	Filters like spec, manual, guide, etc.
Show results accordions expanded.	Yes/NO

---

5. Architecture Flow

1. Author configures block (SKU, document types, etc.)
2. EDS block reads authored values
3. Block creates a container `<div>`
4. Block waits (lazy load via IntersectionObserver)
5. External JS is loaded dynamically

6. Configuration object is created
7. Global function is invoked:

```
window.documentsConfiguration(config)
```

8. External app:
   - Fetches data
   - Renders UI into the container